

# **ONLINE SHOPPING SYSTEM**

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the  
award of the degree of

## **BACHELOR OF TECHNOLOGY**

in

## **ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

Submitted By

**D GIRI BABU – 25B11AI242**

**E DHARANI – 25B11AI291**

**CH SUJATHA – 25B11AI211**

**M VIVEK SURYA – 25B11AI773**

Under the esteemed guidance of

**Mr. T.Srinivasulu M.Tech, (Ph.D.)**

**Assistant Professor**



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE  
LEARNING**

**ADITYA UNIVERSITY**

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

**2026 – 2027**

# ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

## DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



### CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “**ONLINE SHOPPING SYSTEM**” is being submitted by D.Giri Babu (Roll No. 25B11AI242), E Dharani (Roll No. 25B11AI291), Ch Sujatha (Roll No. 25B11AI211) and M Vivek Surya (Roll No.25B11AI773) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

#### Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)  
Associate professor  
HOD

#### Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.  
Associate Professor &

**External Examiner**

## **DECLARATION**

We hereby declare that the DBMS Capstone Project entitled “**ONLINE SHOPPING SYSTEM**” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of

**Mr T Srinivasulu**, Department of **Artificial Intelligence and Machine Learning**, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

### **Project Associates**

D Giri Babu (25B11AI242)

E Dharani (25B11AI291)

Ch Sujatha (25B11AI211)

M Vivek Surya (25B11AI773)

Date: 20-09-2026

Place: Surampalem

## ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide

**Mr T Srinivasulu**, Department of **Artificial Intelligence and Machine Learning**, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to **Dr K Naga Bhargavi**, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

### **Project Associates**

D Giri Babu (25B11AI242)

E Dharani (25B11AI291)

Ch Sujatha (25B11AI211)

M Vivek Surya (25B11AI773)

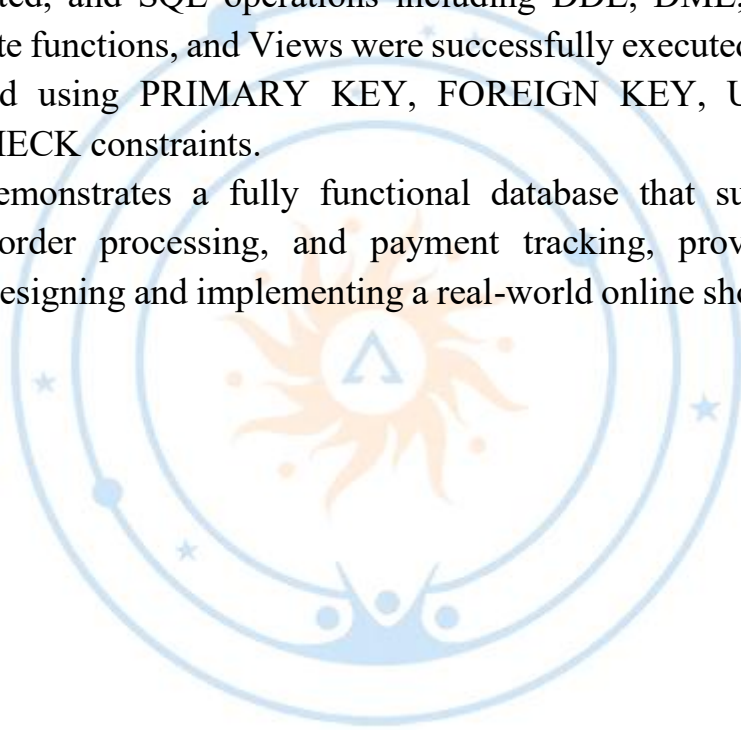
## ABSTRACT

This project presents the complete design and implementation of a relational database for an Online Shopping System. The system efficiently manages customers, products, categories, orders, order items, and payments through a well-normalized schema.

In the design phase, six core entities were identified, attributes were defined, relationships and cardinalities were established, and a complete Entity-Relationship (ER) diagram was developed. The ER model was mapped to a relational schema with Primary Keys, Foreign Keys, and integrity constraints.

In the implementation phase, the database was created in PostgreSQL. Sample data was inserted, and SQL operations including DDL, DML, DQL, CRUD, Joins, Aggregate functions, and Views were successfully executed. Data integrity was maintained using PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints.

The project demonstrates a fully functional database that supports product management, order processing, and payment tracking, providing practical experience in designing and implementing a real-world online shopping database system.



## LIST OF FIGURES

| Figure No. | Title of the Figure                            | Page No. |
|------------|------------------------------------------------|----------|
| Fig. 3.1   | ER Diagram of the Online Shopping System       | 8        |
| Fig. 4.1   | Successful Creation of Tables                  | 15       |
| Fig. 4.2   | Sample Data Inserted into the Tables           | 22       |
| Fig. 4.3   | Referential Integrity Enforced by the Database | 24       |
| Fig. 7.1   | Product Catalog with Category (INNER JOIN)     | 34       |
| Fig. 7.2   | Order Summary with Aggregate Functions         | 34       |

## LIST OF TABLES

| Table No. | Title of the Table                          | Page No. |
|-----------|---------------------------------------------|----------|
| Table 2.1 | Tools and Technologies Used                 | 4        |
| Table 2.2 | Software and Hardware Requirements          | 5        |
| Table 3.1 | Entities Identified for the Selected Domain | 6        |
| Table 3.2 | Attributes of Each Entity                   | 7        |
| Table 3.3 | Relationships and Cardinality               | 9        |
| Table 3.4 | Relational Database Schema                  | 9        |
| Table 3.5 | Primary Key and Foreign Key Definitions     | 11       |
| Table 3.6 | Normalization Steps                         | 11       |
| Table 4.1 | Sample Data Summary                         | 16       |
| Table 4.2 | Constraints and Data Integrity Rules        | 23       |
| Table 6.1 | Modules and Their Functionalities           | 33       |
| Table 8.1 | Test Cases and Results                      | 35       |
| Table 9.1 | Team Responsibilities                       | 36       |
| Table 9.2 | Review-wise Progress Record                 | 36       |

## INDEX

| Chapter  | Title                                     | Page No. |
|----------|-------------------------------------------|----------|
|          | Certificate                               | i        |
|          | Declaration                               | ii       |
|          | Acknowledgement                           | iii      |
|          | Abstract                                  | iv       |
|          | List of Figures                           | v        |
|          | List of Tables                            | v        |
| <b>1</b> | <b>Introduction</b>                       |          |
|          | 1.1 Introduction to the Domain            | 1        |
|          | 1.2 Problem Statement                     | 1        |
|          | 1.3 Motivation and Need for a Database    | 2        |
|          | 1.4 Objectives of the Project             | 2        |
|          | 1.5 Scope of the Project                  | 2        |
|          | 1.6 Organization of the Report            | 3        |
| <b>2</b> | <b>System Study and Tools</b>             |          |
|          | 2.1 Existing System and Its Limitations   | 4        |
|          | 2.2 Proposed Database System              | 4        |
|          | 2.3 Tools and Technologies Used           | 4        |
|          | 2.4 Software and Hardware Requirements    | 5        |
| <b>3</b> | <b>Database Design (Review – I)</b>       |          |
|          | 3.1 Identified Entities                   | 6        |
|          | 3.2 Entity Attributes and Attribute Types | 7        |
|          | 3.3 ER Diagram                            | 8        |
|          | 3.4 Relationships and Cardinality         | 9        |
|          | 3.5 Relational Database Schema            | 9-10     |
|          | 3.6 Primary Keys and Foreign Keys         | 11       |

|           |                                                 |       |
|-----------|-------------------------------------------------|-------|
|           | 3.7 Normalization up to 3NF                     | 11-12 |
| <b>4</b>  | <b>Database Implementation (Review – II)</b>    |       |
|           | 4.1 Database and Table Creation – DDL           | 13-15 |
|           | 4.2 Sample Data Insertion – DML                 | 16-22 |
|           | 4.3 Constraints and Data Integrity              | 23    |
|           | 4.4 Data Validation Rules                       | 24    |
| <b>5</b>  | <b>SQL Operations and Queries</b>               |       |
|           | 5.1 DQL – Basic Retrieval Queries               | 25    |
|           | 5.2 Join Operations                             | 26-28 |
|           | 5.3 Aggregate Functions, GROUP BY and HAVING    | 29    |
|           | 5.4 Nested and Correlated Subqueries            | 30    |
|           | 5.5 Views and Relational Set Operations         | 30-31 |
|           | 5.6 DCL and TCL Operations                      | 32    |
| <b>6</b>  | <b>Modules and Functionalities</b>              | 33    |
| <b>7</b>  | <b>Results – Sample Outputs and Screenshots</b> | 34    |
| <b>8</b>  | <b>Testing and Data Validation</b>              | 35    |
| <b>9</b>  | <b>GitHub Repository and Team Contribution</b>  | 36    |
| <b>10</b> | <b>Conclusion and Future Enhancements</b>       | 37    |
|           | References                                      | 38    |
|           | Appendix A – Complete SQL Script                | 39    |

# CHAPTER 1: INTRODUCTION

## 1.1 Introduction to the Domain

Online shopping is a convenient way of purchasing products and services through the internet. Customers can browse products, compare prices, place orders, make payments, and track their purchases without visiting a physical store. The rapid growth of e-commerce has transformed the retail industry, making digital platforms essential for both businesses and consumers.

An Online Shopping System helps manage these activities in an organised and efficient manner. It provides a platform for customers, products organised by category, multi-item orders, payment records and inventory management, making the shopping process faster and more reliable. Every one of these activities produces data, and the data items are strongly related to one another: a product belongs to a category; an order is placed by a customer and contains several order items; a payment is linked to exactly one order.

The Online Shopping System is a relational database designed to hold this transactional data for one e-commerce store. It covers the customers who register, the categories that organise the catalogue, the products offered for sale, the orders placed by customers, the individual line items within each order, and the payments recorded against those orders.

## 1.2 Problem Statement

At present, traditional retail and poorly designed e-commerce platforms maintain customer lists, product inventories, order sheets and payment records in separate spreadsheets or disconnected files. This arrangement causes four recurring problems.

- Data redundancy — the same customer name, email or product description is re-entered in the customer sheet, in every order register and again in the payment log.
- Data inconsistency — when a product price is corrected in one file, other files continue to hold the old value, leading to wrong invoices.
- No integrity checking — nothing prevents an order being recorded for a customer who does not exist, or a payment being linked to an order that has been deleted.
- Difficult retrieval — a question such as “what is the total revenue per category” or “which products are low on stock” has to be compiled by hand from several files.

The proposed system stores each fact exactly once in a normalised table, links the tables through foreign keys so that invalid data is rejected by the database itself, and answers the above questions with a single SQL statement.

### 1.3 Motivation and Need for a Database

A spreadsheet can store rows, but it cannot enforce relationships. It will happily accept a duplicate email, a negative stock quantity, or an order row pointing at a customer that has been deleted. A database management system enforces these rules at the point of insertion through PRIMARY KEY, UNIQUE, CHECK and FOREIGN KEY constraints, so bad data never enters the system in the first place. It also allows several users to work on the same data concurrently, supports transactions so that a partially completed update can be rolled back, and provides a query language in which complex e-commerce questions are expressed declaratively rather than compiled by hand.

### 1.4 Objectives of the Project

To design a normalized relational database for the selected domain with at least six entities.

To implement the database with appropriate primary keys, foreign keys and integrity constraints.

To implement DML scripts and complex SQL joins to support order processing and multi-table data retrieval.

To develop database views and aggregate queries for product catalogue reports, order statistics and payment tracking.

To verify data integrity by demonstrating that the database rejects duplicate keys, out-of-range values and orphan records through systematic test cases.

### 1.5 Scope of the Project

The database covers the core operations of an online shopping platform: customers, categories, products, orders, order items and payments. It supports product catalogue management, multi-item order processing, stock tracking and payment status recording.

The following are outside the scope of the present work: a graphical front-end web application, user authentication and role-based access control beyond basic DCL examples, real-time payment gateway integration, product recommendation engines, and advanced analytics dashboards. The project is a database design and implementation exercise, and all interaction is through SQL.

## 1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.



## CHAPTER 2: SYSTEM STUDY AND TOOLS

### 2.1 Existing System and Its Limitations

The existing arrangement is often file-based or relies on poorly structured applications. Each section of the store maintains its own spreadsheet, and the files are exchanged by email or shared folders. During the study of the present system the following limitations were observed.

- The same customer and product details appear in at least three different files, so an update has to be applied three times and is often applied only once.
- There is no automatic link between an order and a payment; the finance team matches them by order number, and a typing mistake produces an orphan payment record.
- Range checks are absent — prices, stock quantities and order amounts are typed as free text, allowing negative values.
- Concurrent editing is not possible; only one person can hold the master file at a time.
- Summary reports such as category-wise revenue or low-stock alerts must be prepared manually for every request.

### 2.2 Proposed Database System

The proposed system is a single relational database in which each fact is stored once. Customer, Category, Product and Payment details are held in their own tables; the many-to-many association between orders and products is resolved by the OrderItem table, and each Order carries exactly one Payment row (1:1). Foreign keys make orphan records impossible, CHECK constraints keep prices and quantities within valid ranges, and UNIQUE constraints prevent duplicate emails and category names. Reports that were previously manual are obtained by a single SELECT statement, and frequently used reports are stored as views.

### 2.3 Tools and Technologies Used

**Table 2.1: Tools and Technologies Used**

| Category           | Tool / Technology Used | Version / Remarks                       |
|--------------------|------------------------|-----------------------------------------|
| Database Server    | PostgreSQL             | Relational DBMS used for implementation |
| Query / Admin Tool | pgAdmin / psql         | Script execution and output capture     |
| ER Diagram Tool    | draw.io (diagrams.net) | Web version — ER diagram                |
| Version Control    | Git and GitHub         | Team repository for the SQL script      |
| Documentation      | Microsoft Word         | Report preparation                      |
| Presentation       | Microsoft PowerPoint   | Review presentations                    |

## 2.4 Software and Hardware Requirements

**Table 2.2: Software and Hardware Requirements**

| Requirement Type | Specification                                     |
|------------------|---------------------------------------------------|
| Operating System | Windows 10 / 11 (64-bit) or Ubuntu 22.04          |
| Database Server  | PostgreSQL 14 or above                            |
| Client Tool      | pgAdmin / psql command-line client                |
| Processor        | Intel Core i3 or above                            |
| RAM              | 4 GB minimum, 8 GB recommended                    |
| Disk Space       | 2 GB free (server, sample data and documentation) |



## CHAPTER 3: DATABASE DESIGN

*(Deliverables of Review – I)*

### 3.1 Identified Entities

**Table 3.1: Entities Identified for the Selected Domain**

| S.No | Entity Name | Description                                                                                                     |
|------|-------------|-----------------------------------------------------------------------------------------------------------------|
| 1    | Customer    | A registered user who browses products, places orders and makes payments. Stores personal and contact details.. |
| 2    | Category    | A logical grouping of products (e.g., Electronics, Clothing, Books) that helps organize the catalog.            |
| 3    | Product     | An item available for sale in the store. Contains name, price, stock quantity and category link.                |
| 4    | Orders      | A purchase request placed by a customer. Records date, status and total amount.                                 |
| 5    | Order Item  | Represents each product line inside an order — quantity ordered and price at the time of purchase.              |
| 6    | Payment     | Records the payment made against an order — amount, method, date and status.                                    |

### 3.2 Entity Attributes and Attribute Types

**Table 3.2: Attributes of Each Entity**

| Entity     | Attribute                                                        | Data Type                                | Key<br>(PK/FK/—)        | Attribute Type                                  |
|------------|------------------------------------------------------------------|------------------------------------------|-------------------------|-------------------------------------------------|
| Customer   | Customer ID<br>Name,Email,Phone<br>Address,Password              | INT<br>VARCHAR<br>VARCHAR                | PK<br>-<br>-            | Simple<br>Simple<br>simple                      |
| Category   | Category ID<br>CategoryName,,Description                         | INT<br>VARCHAR                           | PK<br>-                 | Simple<br>simple                                |
| Product    | Product ID<br>Name,Description<br>Price,stock<br>Category id     | INT<br>VARCHAR<br>DECIMAL/INT<br>INT     | PK<br>-<br>-<br>FK      | Simple<br>Simple<br>Simple<br>simple            |
| Orders     | Order ID<br>Customer id<br>Order date,status<br>Total amount     | INT<br>INT<br>DATE/VARCHAR<br>DECIMAL    | PK<br>FK<br>-<br>-<br>- | Simple<br>Simple<br>Simple<br>Simple<br>Derived |
| Order Item | Order ItemID<br>Order id,product id<br>Quantity<br>unitprice     | INT<br>INT<br>INT<br>DECIMAL             | PK<br>FK<br>-<br>-      | Simple<br>Simple<br>Simple<br>Simple            |
| Payment    | PaymentID<br>Order id<br>Amount<br>Method,status<br>Payment Date | INT<br>INT<br>DECIMAL<br>VARCHAR<br>DATE | PK<br>FK<br>-<br>-<br>- | Simple<br>Simple<br>Simple<br>Simple<br>simple  |

### 3.3 ER Diagram

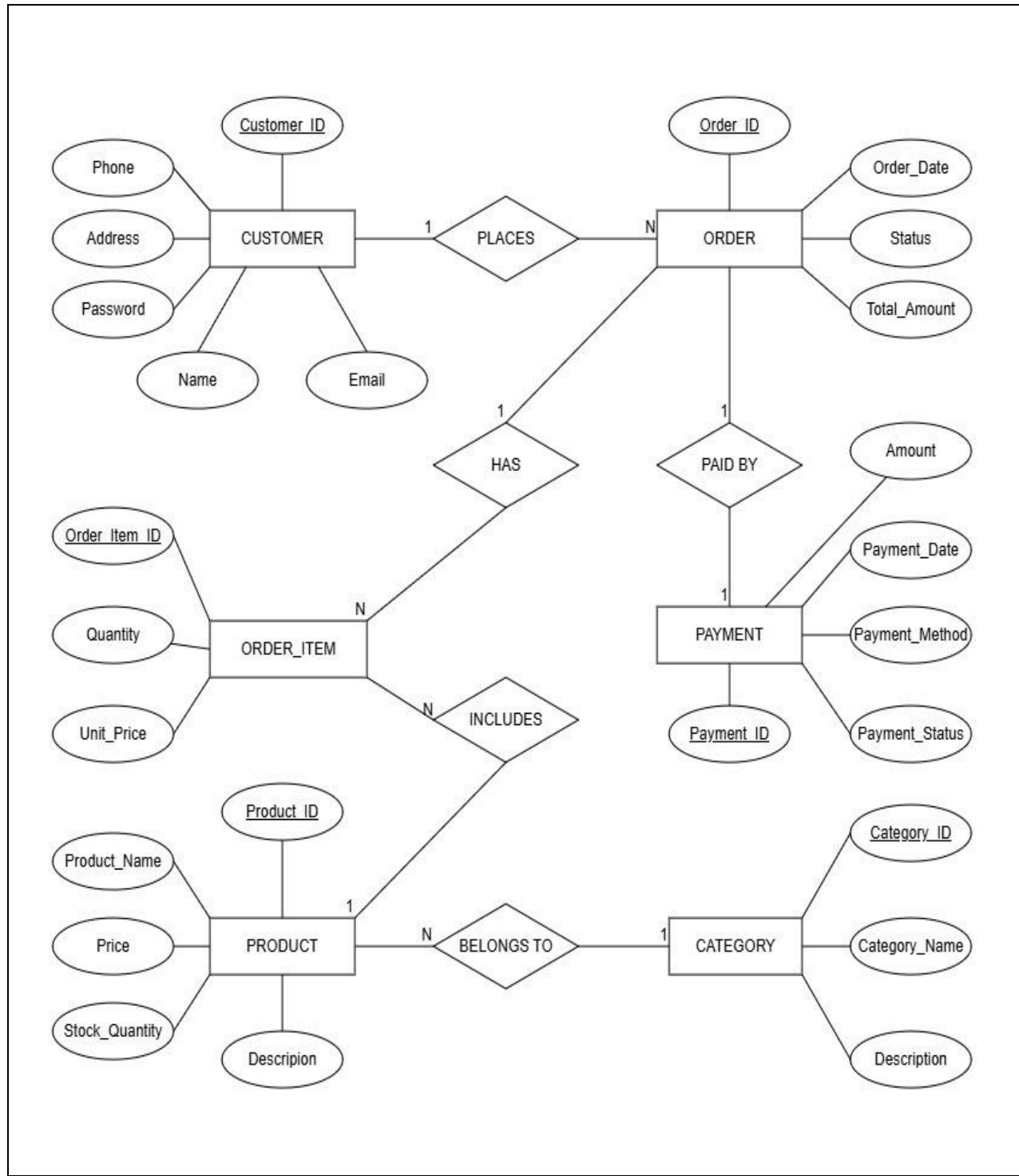


Fig. 3.1: ER Diagram of the Online Shopping System

### 3.4 Relationships and Cardinality

**Table 3.3: Relationships and Cardinality**

| Entity 1 | Relationship | Entity 2   | Cardinality<br>(1:1 / 1:N /<br>M:N) | Participation       |
|----------|--------------|------------|-------------------------------------|---------------------|
| CUSTOMER | PLACES       | ORDER      | 1:N                                 | Partial on Customer |
| ORDER    | HAS          | ORDER_ITEM | 1:N                                 | Total on both sides |
| ORDER    | PAID BY      | PAYMENT    | 1:1                                 | Partial on Order    |
| PRODUCT  | INCLUDES     | ORDER_ITEM | 1:N                                 | Partial on Product  |
| CATEGORY | BELONGS TO   | PRODUCT    | 1:N                                 | Total on Product    |

### 3.5 Relational Database Schema

**Table 3.4: Relational Database Schema**

| Table Name | Column Name   | Data Type | Key (PK / FK<br>/ —) | Constraints                        |
|------------|---------------|-----------|----------------------|------------------------------------|
| Category   | Category ID   | INT       | PK                   | Primary key,<br>auto_increment     |
| Category   | Category Name | VARCHAR   | -                    | Unique, not null                   |
| Category   | Description   | TEXT      | -                    | None                               |
| Customer   | CustomerID    | INT       | PK                   | Primary key,<br>auto_increment     |
| Customer   | Email         | VARCHAR   | -                    | Unique, not null                   |
| Product    | ProductID     | INT       | PK                   | Primary key,<br>auto_increment     |
| Product    | Price         | DECIMAL   | -                    | CHECK (Price ≥ 0)                  |
| Product    | CategoryID    | INT       | FK                   | REFERENCES<br>Category(categoryid) |

## Schema Diagram

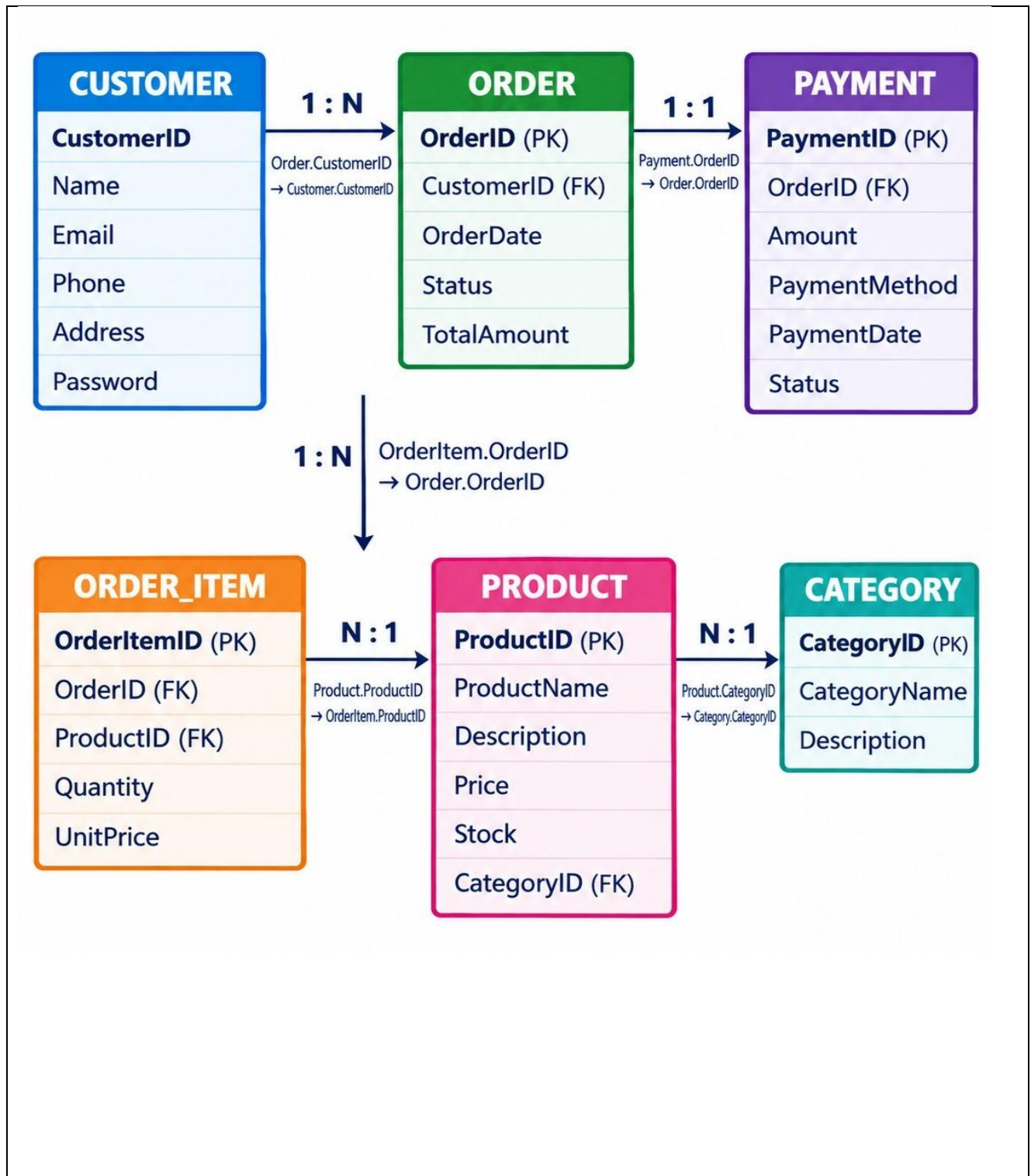


Fig. 3.2: Relational Schema Diagram

### 3.6 Primary Keys and Foreign Keys

**Table 3.5: Primary Key and Foreign Key Definitions**

| Table     | Primary Key | Foreign Key        | References<br>(Table.Column)          |
|-----------|-------------|--------------------|---------------------------------------|
| Customer  | CustomerID  | —                  | —                                     |
| Category  | CategoryID  | —                  | —                                     |
| Product   | ProductID   | CategoryID         | Category(CategoryID)                  |
| Order     | OrderID     | CustomerID         | Customer(CustomerID)                  |
| OrderItem | OrderItemID | OrderID, ProductID | Order(OrderID),<br>Product(ProductID) |
| Payment   | PaymentID   | OrderID            | Order(OrderID)                        |

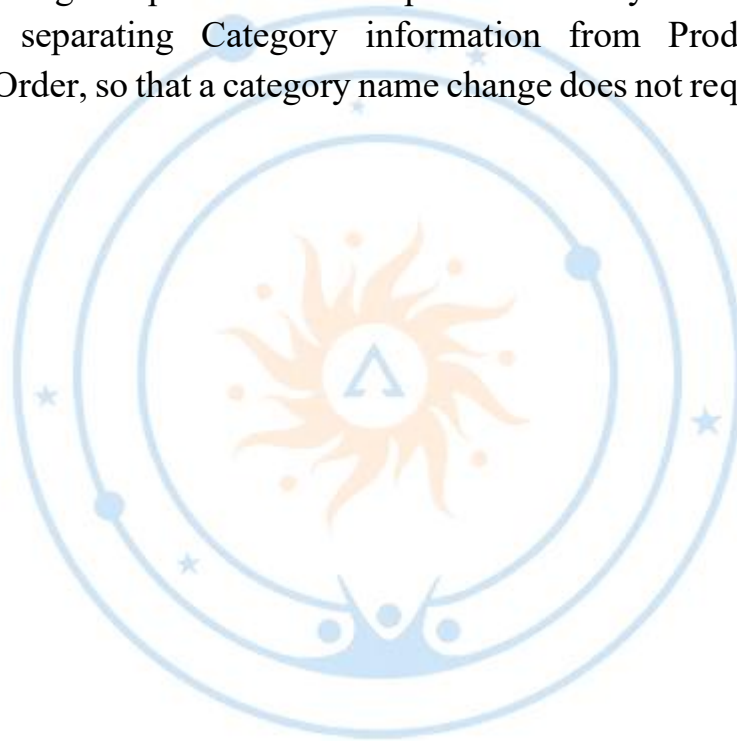
### 3.7 Normalization up to 3NF

The Online Shopping System database was normalised up to 3NF to reduce data redundancy and update anomalies.

**Table 3.6: Normalization Steps**

| Normal Form | Condition Satisfied                                       | Anomaly Removed                            | Resulting Tables                    |
|-------------|-----------------------------------------------------------|--------------------------------------------|-------------------------------------|
| UNF         | All customer, product, order and payment details together | —                                          | Single flat relation                |
| 1NF         | Removed repeating groups; all values atomic               | Repeating groups / multi-valued attributes | Atomic columns                      |
| 2NF         | Removed partial dependencies                              | Partial dependency anomalies               | Customer, Product, Order, OrderItem |
| 3NF         | Removed transitive dependencies                           | Transitive dependency anomalies            | Category, Payment separated         |

UNF contained all customer, product, order and payment details together in a single flat relation. 1NF removed repeating groups (for example multiple products listed in one order row) and ensured all values are atomic. 2NF removed partial dependencies by separating Customer, Product, Order and OrderItem into independent tables so that product details no longer depended on a composite order key. 3NF removed transitive dependencies by separating Category information from Product and Payment information from Order, so that a category name change does not require updating every product row.



## CHAPTER 4: DATABASE IMPLEMENTATION

*(Deliverables of Review – II)*

### 4.1 Database and Table Creation – DDL

```
DROP TABLE IF EXISTS Payment, OrderItem, "Order", Product, Customer, Category  
CASCADE;
```

```
-- =====  
-- 2. CREATE CATEGORY TABLE  
-- =====
```

```
CREATE TABLE Category (  
    CategoryID SERIAL PRIMARY KEY,  
    CategoryName VARCHAR(100) NOT NULL UNIQUE,  
    Description VARCHAR(255)  
);
```

```
-- =====  
-- 3. CREATE CUSTOMER TABLE  
-- =====
```

```
CREATE TABLE Customer (  
    CustomerID SERIAL PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) NOT NULL UNIQUE,  
    Phone VARCHAR(15) UNIQUE,  
    Address VARCHAR(255),  
    Password VARCHAR(100) NOT NULL  
);
```

```
-- =====  
-- 4. CREATE PRODUCT TABLE  
-- =====
```

```
CREATE TABLE Product (  
    ProductID SERIAL PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Description VARCHAR(255),  
    Price DECIMAL(10,2) NOT NULL CHECK (Price >= 0),  
    Stock INT NOT NULL CHECK (Stock >= 0),
```

```

CategoryID INT NOT NULL,

CONSTRAINT fk_product_category
    FOREIGN KEY (CategoryID)
    REFERENCES Category(CategoryID)
);

-- =====
-- 5. CREATE ORDER TABLE
-- =====

CREATE TABLE "Order" (
    OrderID SERIAL PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATE NOT NULL,
    OrderTime TIME NOT NULL,
    Status VARCHAR(50) NOT NULL,
    TotalAmount DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),

    CONSTRAINT fk_order_customer
        FOREIGN KEY (CustomerID)
        REFERENCES Customer(CustomerID)
);

-- =====
-- 6. CREATE ORDER ITEM TABLE
-- =====

CREATE TABLE OrderItem (
    OrderItemID SERIAL PRIMARY KEY,
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice >= 0),

    CONSTRAINT fk_orderitem_order
        FOREIGN KEY (OrderID)
        REFERENCES "Order"(OrderID),

    CONSTRAINT fk_orderitem_product
        FOREIGN KEY (ProductID)
        REFERENCES Product(ProductID)
);

```

```
-- =====
-- 7. CREATE PAYMENT TABLE
-- =====
```

```
CREATE TABLE Payment (
  PaymentID SERIAL PRIMARY KEY,
  OrderID INT NOT NULL,
  Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
  Method VARCHAR(50) NOT NULL,
  PaymentDate DATE NOT NULL,
  PaymentTime TIME NOT NULL,
  Status VARCHAR(50) NOT NULL,

  CONSTRAINT fk_payment_order
  FOREIGN KEY (OrderID)
  REFERENCES "Order"(OrderID)
);
```

The script executed without errors and created seven tables. The output of SHOW TABLES is shown below.

| Tables_in<br>Online Shopping System |
|-------------------------------------|
| Customer                            |
| Category                            |
| Product                             |
| Order                               |
| Order Item                          |
| Payment                             |

**Fig. 4.1: Successful Creation of Tables**

## 4.2 Sample Data Insertion – DML

Table 4.1: Sample Data Summary

| S.No | Table Name | No. of Records Inserted | Sample Data                                                                      |
|------|------------|-------------------------|----------------------------------------------------------------------------------|
| 1    | Category   | 5                       | Electronics, Clothing, Books, Home appliances, Sports                            |
| 2    | Customer   | 6                       | Rahul Sharma, Priya patel, Amit Kumar, Sneha Reddy, Vikram Singh, Ananya Guptha, |
| 3    | Product    | 12                      | Wireless Headphones, Cotton T-Shirt, Coffee Maker, Formal Shirt                  |
| 4    | order      | 7                       | OrderID1 (Delivered), OrderID 4(Confirmed), OrderID 5 (Pending)                  |
| 5    | orderitem  | 11                      | (Order 1, Iphone 15), (Order1, Wireless Earbuds), (Order 7, Women Summer Dress)  |
| 6    | payment    | 7                       | UPI , Credit Card, Net banking, Debit Card. Cash on Delivery                     |

```
-- =====  
-- 8. INSERT CATEGORY DATA  
-- =====
```

```
INSERT INTO Category  
(CategoryName, Description)  
VALUES  
( 'Electronics',  
  'Mobile phones, laptops, gadgets and accessories'),  
  
( 'Clothing',  
  'Men and Women apparel, footwear and fashion'),  
  
( 'Books',  
  'Fiction, Non-fiction, Academic and Competitive books'),  
  
( 'Home Appliances',  
  'Kitchen appliances and home essentials'),  
  
( 'Sports',  
  'Sports equipment and fitness accessories');
```

```
-- =====  
-- 9. INSERT CUSTOMER DATA  
-- =====
```

```
INSERT INTO Customer  
(Name, Email, Phone, Address, Password)  
VALUES
```

```
('Rahul Sharma',  
'rahul.sharma@gmail.com',  
'9876543210',  
'Hyderabad, Telangana',  
'rahul@123'),
```

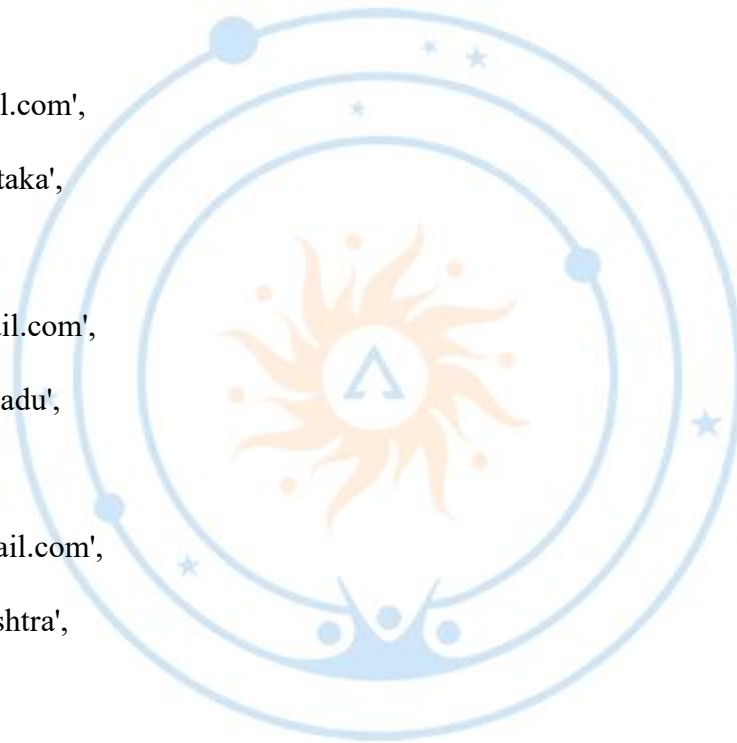
```
('Priya Patel',  
'priya.patel@gmail.com',  
'9876543211',  
'Bangalore, Karnataka',  
'priya@456'),
```

```
('Amit Kumar',  
'amit.kumar@gmail.com',  
'9876543212',  
'Chennai, Tamil Nadu',  
'amit@789'),
```

```
('Sneha Reddy',  
'sneha.reddy@gmail.com',  
'9876543213',  
'Mumbai, Maharashtra',  
'sneha@321'),
```

```
('Vikram Singh',  
'vikram.singh@gmail.com',  
'9876543214',  
'Delhi',  
'vikram@654'),
```

```
('Ananya Gupta',  
'ananya.gupta@gmail.com',  
'9876543215',  
'Pune, Maharashtra',  
'ananya@987');
```



-- 10. INSERT PRODUCT DATA

-- =====

INSERT INTO Product

(Name, Description, Price, Stock, CategoryID)

VALUES

('iPhone 15',

'Latest Apple smartphone with A16 chip',

79999.00, 25, 1),

('Samsung Galaxy S24',

'Flagship Android phone with AI features',

74999.00, 30, 1),

('Sony WH-1000XM5',

'Noise cancelling wireless headphones',

29990.00, 40, 1),

('Men Formal Shirt',

'Premium cotton formal shirt',

1299.00, 100, 2),

('Women Summer Dress',

'Light and comfortable summer dress',

1899.00, 80, 2),

('Python Programming',

'Complete guide to Python programming',

599.00, 50, 3),

('Atomic Habits',

'Book by James Clear on building habits',

450.00, 60, 3),

('Air Fryer 4L',

'Healthy oil-free cooking appliance',

4999.00, 35, 4),

('Mixer Grinder',

'750W powerful mixer grinder',

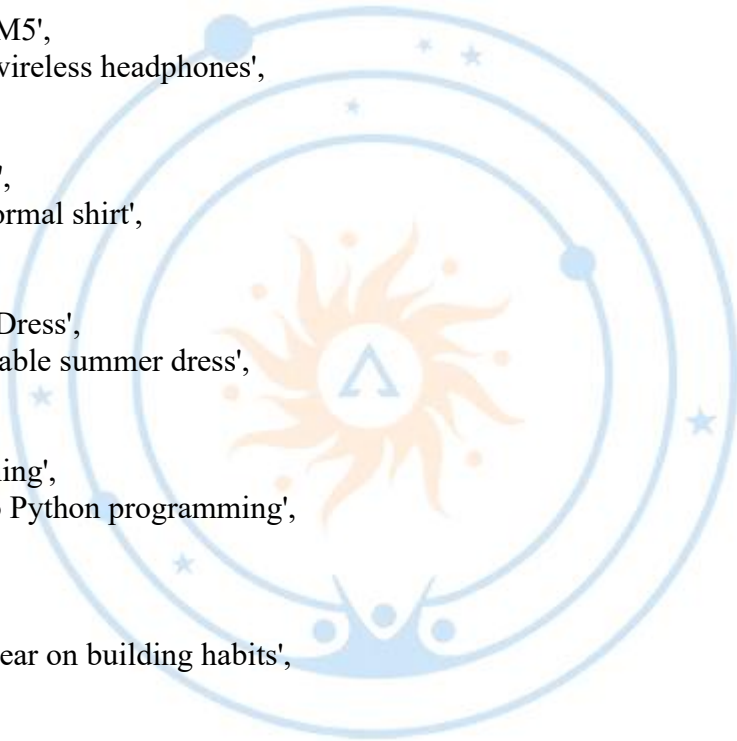
3499.00, 45, 4),

('Yoga Mat',

'Anti-slip yoga mat for fitness',

899.00, 70, 5),

('Dumbbell Set 10kg',



'Adjustable dumbbell set',  
2499.00, 25, 5),

('Wireless Earbuds',  
'Bluetooth earbuds with long battery',  
2999.00, 90, 1);

---

---

-- 11. INSERT ORDER DATA

---

---

INSERT INTO "Order"  
(CustomerID, OrderDate, OrderTime, Status, TotalAmount)  
VALUES

(1, '2025-08-15', '10:30:00',  
'Delivered', 82998.00),

(2, '2025-08-20', '14:15:00',  
'Delivered', 1299.00),

(3, '2025-08-25', '09:00:00',  
'Shipped', 5598.00),

(1, '2025-09-01', '16:45:00',  
'Confirmed', 2999.00),

(4, '2025-09-05', '11:20:00',  
'Pending', 4999.00),

(5, '2025-09-08', '13:10:00',  
'Confirmed', 3398.00),

(6, '2025-09-10', '17:30:00',  
'Pending', 1899.00);

---

---

-- 12. INSERT ORDER ITEM DATA

---

---

INSERT INTO OrderItem  
(OrderID, ProductID, Quantity, UnitPrice)  
VALUES

(1, 1, 1, 79999.00),

(1, 12, 1, 2999.00),

(2, 4, 1, 1299.00),

(3, 8, 1, 4999.00),

(3, 6, 1, 599.00),

(4, 12, 1, 2999.00),

(5, 8, 1, 4999.00),

(6, 4, 1, 1299.00),

(6, 10, 1, 899.00),

(6, 7, 1, 450.00),

(7, 5, 1, 1899.00);

---

---

-- 13. INSERT PAYMENT DATA

---

---

INSERT INTO Payment  
(OrderID, Amount, Method, PaymentDate, PaymentTime, Status)  
VALUES

(1, 82998.00, 'UPI',  
'2025-08-15', '10:35:00', 'Completed'),

(2, 1299.00, 'Credit Card',  
'2025-08-20', '14:20:00', 'Completed'),

(3, 5598.00, 'Net Banking',  
'2025-08-25', '09:15:00', 'Completed'),

(4, 2999.00, 'UPI',  
'2025-09-01', '16:50:00', 'Completed'),

(5, 4999.00, 'Debit Card',  
'2025-09-05', '11:25:00', 'Pending'),

(6, 3398.00, 'UPI',  
'2025-09-08', '13:15:00', 'Completed'),

(7, 1899.00, 'Cash on Delivery',  
'2025-09-10', '17:35:00', 'Pending');

```

-- =====
-- 14. DISPLAY CATEGORY TABLE
-- =====

SELECT * FROM Category;

-- =====
-- 15. DISPLAY CUSTOMER TABLE
-- =====

SELECT * FROM Customer;

-- =====
-- 16. DISPLAY PRODUCT TABLE
-- =====

SELECT * FROM Product;

-- =====
-- 17. DISPLAY ORDER TABLE
-- DATE FORMAT: DD-MM-YYYY
-- TIME FORMAT: HH:MM:SS
-- =====

SELECT
    OrderID,
    CustomerID,
    TO_CHAR(OrderDate, 'DD-MM-YYYY') AS OrderDate,
    TO_CHAR(OrderTime, 'HH24:MI:SS') AS OrderTime,
    Status,
    TotalAmount
FROM "Order";

-- =====
-- 18. DISPLAY ORDER ITEM TABLE
-- =====

SELECT * FROM OrderItem;

-- =====
-- 19. DISPLAY PAYMENT TABLE

```

```
-- DATE FORMAT: DD-MM-YYYY
```

```
-- TIME FORMAT: HH:MM:SS
```

```
-- =====
```

```
SELECT
```

```
    PaymentID,
```

```
    OrderID,
```

```
    Amount,
```

```
    Method,
```

```
    TO_CHAR(PaymentDate, 'DD-MM-YYYY') AS PaymentDate,
```

```
    TO_CHAR(PaymentTime, 'HH24:MI:SS') AS PaymentTime,
```

```
    Status
```

```
FROM Payment;
```



| customerid | name         | email                  | phone      | address              | password   |
|------------|--------------|------------------------|------------|----------------------|------------|
| 1          | Rahul Sharma | rahul.sharma@gmail.com | 9876543210 | Hyderabad, Telangana | rahul@123  |
| 2          | Priya Patel  | priya.patel@gmail.com  | 9876543211 | Bangalore, Karnataka | priya@456  |
| 3          | Amit Kumar   | amit.kumar@gmail.com   | 9876543212 | Chennai, Tamil Nadu  | amit@789   |
| 4          | Sneha Reddy  | sneha.reddy@gmail.com  | 9876543213 | Mumbai, Maharashtra  | sneha@321  |
| 5          | Vikram Singh | vikram.singh@gmail.com | 9876543214 | Delhi                | vikram@654 |
| 6          | Ananya Gupta | ananya.gupta@gmail.com | 9876543215 | Pune, Maharashtra    | ananya@987 |

| productid | name               | description                             | price    | stock | categoryid |
|-----------|--------------------|-----------------------------------------|----------|-------|------------|
| 1         | iPhone 15          | Latest Apple smartphone with A16 chip   | 79999.00 | 25    | 1          |
| 2         | Samsung Galaxy S24 | Flagship Android phone with AI features | 74999.00 | 30    | 1          |
| 3         | Sony WH-1000XM5    | Noise cancelling wireless headphones    | 29990.00 | 40    | 1          |
| 4         | Men Formal Shirt   | Premium cotton formal shirt             | 1299.00  | 100   | 2          |
| 5         | Women Summer Dress | Light and comfortable summer dress      | 1899.00  | 80    | 2          |

**Fig. 4.2: Sample Data Inseted into the Tables**

### 4.3 Constraints and Data Integrity

**Table 4.2: Constraints and Data Integrity Rules**

| Constraint / Validation | Applied On (Table.Column)                              | Purpose                                                                                                 | Evidence / Screenshot Ref.                                                                                                                             |
|-------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRIMARY KEY             | All ID columns                                         | Ensures each record has a unique identifier and prevents duplicate IDs.                                 | CategoryID SERIAL PRIMARY KEY, CustomerID SERIAL PRIMARY KEY, OrderID SERIAL PRIMARY KEY, OrderItemID SERIAL PRIMARY KEY, PaymentID SERIAL PRIMARY KEY |
| UNIQUE                  | IDs, Email                                             | Prevents duplicate values.                                                                              | CategoryName VARCHAR(100) NOT NULL UNIQUE and Email VARCHAR(100) NOT NULL UNIQUE                                                                       |
| FOREIGN KEY             | CategoryID, CustomerID, OrderID, ProductID             | Maintains relationships between tables.                                                                 | FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID) and similar relationships for Order, Customer and Product                                     |
| CHECK                   | Price, Stock, Quantity, Amount                         | Prevents invalid values such as negative prices, stock, and order amounts and Allows only valid values. | CHECK (Price >= 0), CHECK (Stock >= 0), CHECK (Quantity > 0), CHECK (UnitPrice >= 0), CHECK (TotalAmount >= 0)                                         |
| NOT NULL                | Name, Email, Phone, Price, Quantity, Amount, etc.      | Prevents missing important data.                                                                        | Name VARCHAR(100) NOT NULL, Email VARCHAR(100) NOT NULL, Price DECIMAL(10,2) NOT NULL, Quantity INT NOT NULL, etc.                                     |
| DEFAULT                 | Order.Status, OrderDate, Product.Stock, Payment.Status | Automatically provides a default value when no value is entered.                                        | Status VARCHAR(50) DEFAULT 'Pending'                                                                                                                   |

## 4.4 Data Validation Rules



**(a) Referential integrity — product linked to a category that does not exist:**

SQL

```
INSERT INTO Product (Name, Description, Price, Stock, CategoryID)
VALUES ('Fake Phone', 'Test product', 15000.00, 5, 99);
```

**(b) CHECK constraint — a product with a negative price:**

SQL

```
INSERT INTO Product (Name, Description, Price, Stock, CategoryID)
VALUES ('Invalid Item', 'Negative price test', -500.00, 10, 1);
```

**(c) UNIQUE constraint — a duplicate customer email:**

SQL

```
INSERT INTO Customer (Name, Email, Phone, Address, Password)
VALUES ('Test User', 'rahul.sharma@gmail.com', '9876543210', 'Hyderabad', 'pass123');
```

**Fig. 4.3: Referential Integrity Enforced by the Database**

## CHAPTER 5: SQL OPERATIONS AND QUERIES

### 5.1 DQL – Basic Retrieval Queries

#### Q1. Retrieve all customers.

```
SELECT * FROM Customer;
```

| customerid | name         | email                  | phone      | address              | password   |
|------------|--------------|------------------------|------------|----------------------|------------|
| 1          | Rahul Sharma | rahul.sharma@gmail.com | 9876543210 | Hyderabad, Telangana | rahul@123  |
| 2          | Priya Patel  | priya.patel@gmail.com  | 9876543211 | Bangalore, Karnataka | priya@456  |
| 3          | Amit Kumar   | amit.kumar@gmail.com   | 9876543212 | Chennai, Tamil Nadu  | amit@789   |
| 4          | Sneha Reddy  | sneha.reddy@gmail.com  | 9876543213 | Mumbai, Maharashtra  | sneha@321  |
| 5          | Vikram Singh | vikram.singh@gmail.com | 9876543214 | Delhi                | vikram@654 |
| 6          | Ananya Gupta | ananya.gupta@gmail.com | 9876543215 | Pune, Maharashtra    | ananya@987 |

#### Q2. Products priced above ₹5,000, ordered by price descending.

```
SELECT ProductID, Name, Price, Stock FROM Product  
WHERE Price > 5000 ORDER BY Price DESC;
```

| productid | name               | price    | stock |
|-----------|--------------------|----------|-------|
| 1         | iPhone 15          | 79999.00 | 25    |
| 2         | Samsung Galaxy S24 | 74999.00 | 30    |
| 3         | Sony WH-1000XM5    | 29990.00 | 40    |

#### Q3. Pending payments.

```
SELECT * FROM Payment WHERE Status = 'Pending';
```

| paymentid | orderid | amount  | method           | paymentdate                | paymenttime | status  |
|-----------|---------|---------|------------------|----------------------------|-------------|---------|
| 5         | 5       | 4999.00 | Debit Card       | "2025-09-05T00:00:00.000Z" | 11:25:00    | Pending |
| 7         | 7       | 1899.00 | Cash on Delivery | "2025-09-10T00:00:00.000Z" | 17:35:00    | Pending |

## 5.2 Join Operations

### Q4. INNER JOIN — product details with category names.

```
SELECT p.Name AS ProductName, c.CategoryName, p.Price  
FROM Product p INNER JOIN Category c ON p.CategoryID = c.CategoryID;
```

| productname        | categoryname    | price    |
|--------------------|-----------------|----------|
| iPhone 15          | Electronics     | 79999.00 |
| Samsung Galaxy S24 | Electronics     | 74999.00 |
| Sony WH-1000XM5    | Electronics     | 29990.00 |
| Men Formal Shirt   | Clothing        | 1299.00  |
| Women Summer Dress | Clothing        | 1899.00  |
| Python Programming | Books           | 599.00   |
| Atomic Habits      | Books           | 450.00   |
| Air Fryer 4L       | Home Appliances | 4999.00  |
| Mixer Grinder      | Home Appliances | 3499.00  |
| Yoga Mat           | Sports          | 899.00   |
| Dumbbell Set 10kg  | Sports          | 2499.00  |
| Wireless Earbuds   | Electronics     | 2999.00  |

### Q5. LEFT JOIN — all customers with their orders (including customers who have not ordered).

```
SELECT c.Name, o.OrderID, o.Status
FROM Customer c LEFT JOIN "Order" o ON c.CustomerID = o.CustomerID;
```

| name         | orderid | status    |
|--------------|---------|-----------|
| Rahul Sharma | 1       | Delivered |
| Priya Patel  | 2       | Delivered |
| Amit Kumar   | 3       | Shipped   |
| Rahul Sharma | 4       | Confirmed |
| Sneha Reddy  | 5       | Pending   |
| Vikram Singh | 6       | Confirmed |
| Ananya Gupta | 7       | Pending   |

#### Q6. RIGHT JOIN — all orders with matching customer names.

```
SELECT o.OrderID, c.Name FROM Customer c
RIGHT JOIN "Order" o ON c.CustomerID = o.CustomerID;
```

| orderid | name         |
|---------|--------------|
| 1       | Rahul Sharma |
| 2       | Priya Patel  |
| 3       | Amit Kumar   |
| 4       | Rahul Sharma |
| 5       | Sneha Reddy  |
| 6       | Vikram Singh |
| 7       | Ananya Gupta |

#### Q7. FULL OUTER JOIN — all customers and all orders.

```
SELECT c.Name, o.OrderID FROM Customer c
FULL OUTER JOIN "Order" o ON c.CustomerID = o.CustomerID;
```

| name         | orderid |
|--------------|---------|
| Rahul Sharma | 1       |
| Priya Patel  | 2       |
| Amit Kumar   | 3       |
| Rahul Sharma | 4       |
| Sneha Reddy  | 5       |
| Vikram Singh | 6       |
| Ananya Gupta | 7       |

#### Q8. NATURAL JOIN — Order and Payment on common column OrderID.

```
SELECT * FROM "Order" NATURAL JOIN Payment;
```

| orderid | status  | customerid | orderdate                  | ordertime | totalamount | paymentid | amount  | method           |
|---------|---------|------------|----------------------------|-----------|-------------|-----------|---------|------------------|
| 5       | Pending | 4          | "2025-09-05T00:00:00.000Z" | 11:20:00  | 4999.00     | 5         | 4999.00 | Debit Card       |
| 7       | Pending | 6          | "2025-09-10T00:00:00.000Z" | 17:30:00  | 1899.00     | 7         | 1899.00 | Cash on Delivery |

### 5.3 Aggregate Functions, GROUP BY and HAVING

#### Q9. Overall order statistics.

```
SELECT SUM(TotalAmount), MAX(TotalAmount), COUNT(OrderID),  
       AVG(TotalAmount), MIN(TotalAmount) FROM "Order";
```

| sum       | max      | count | avg                | min     |
|-----------|----------|-------|--------------------|---------|
| 103190.00 | 82998.00 | 7     | 14741.428571428571 | 1299.00 |

#### Q10. Categories with average product price above ₹30.

```
SELECT CategoryID, COUNT(*) AS ProductCount, AVG(Price) AS AvgPrice  
FROM Product GROUP BY CategoryID HAVING AVG(Price) > 30.00;
```

| categoryid | productcount | avgprice              |
|------------|--------------|-----------------------|
| 4          | 2            | 4249.0000000000000000 |
| 2          | 2            | 1599.0000000000000000 |
| 3          | 2            | 524.5000000000000000  |
| 1          | 4            | 46996.750000000000    |
| 5          | 2            | 1699.0000000000000000 |

#### Q11. Inventory metrics across all products.

```
SELECT COUNT(*) AS TotalProducts, AVG(Price) AS AvgPrice,  
       SUM(Stock) AS TotalStock FROM Product;
```

| totalproducts | avgprice           | totalstock |
|---------------|--------------------|------------|
| 12            | 17010.833333333333 | 650        |

## 5.4 Nested and Correlated Subqueries

### Q12. Nested subquery — products priced above the overall average.

```
SELECT Name, Price FROM Product
WHERE Price > (SELECT AVG(Price) FROM Product);
```

| name               | price    |
|--------------------|----------|
| iPhone 15          | 79999.00 |
| Samsung Galaxy S24 | 74999.00 |
| Sony WH-1000XM5    | 29990.00 |

### Q13. Nested subquery with IN — orders placed by customers from Maharashtra.

```
SELECT * FROM "Order"
WHERE CustomerID IN (
    SELECT CustomerID FROM Customer WHERE Address LIKE '%Maharashtra%');
```

| orderid | customerid | orderdate                  | ordertime | status  | totalamount |
|---------|------------|----------------------------|-----------|---------|-------------|
| 5       | 4          | "2025-09-05T00:00:00.000Z" | 11:20:00  | Pending | 4999.00     |
| 7       | 6          | "2025-09-10T00:00:00.000Z" | 17:30:00  | Pending | 1899.00     |

## 5.5 Views and Relational Set Operations

Two views were created. **vw\_ProductCatalog** joins **Product** and **Category**; **vw\_OrderSummary** joins **Order**, **Customer** and **Payment**.

```
CREATE VIEW vw_ProductCatalog AS
SELECT p.Name, p.Price, c.CategoryName
FROM Product p JOIN Category c ON p.CategoryID = c.CategoryID;
```

```
CREATE VIEW vw_OrderSummary AS
SELECT o.OrderID, c.Name AS CustomerName, o.TotalAmount, pay.Status
FROM "Order" o
JOIN Customer c ON o.CustomerID = c.CustomerID
LEFT JOIN Payment pay ON o.OrderID = pay.OrderID;
```

**Q14. Query the product catalogue view.**

```
SELECT * FROM vw_ProductCatalog;
```

| name               | price    | categoryname    |
|--------------------|----------|-----------------|
| iPhone 15          | 79999.00 | Electronics     |
| Samsung Galaxy S24 | 74999.00 | Electronics     |
| Sony WH-1000XM5    | 29990.00 | Electronics     |
| Men Formal Shirt   | 1299.00  | Clothing        |
| Women Summer Dress | 1899.00  | Clothing        |
| Python Programming | 599.00   | Books           |
| Atomic Habits      | 450.00   | Books           |
| Air Fryer 4L       | 4999.00  | Home Appliances |
| Mixer Grinder      | 3499.00  | Home Appliances |
| Yoga Mat           | 899.00   | Sports          |
| Dumbbell Set 10kg  | 2499.00  | Sports          |
| Wireless Earbuds   | 2999.00  | Electronics     |

**Q15. Query the order summary view for completed payments.**

```
SELECT * FROM vw_OrderSummary WHERE Status = 'Completed';
```

| orderid | customername | totalamount | status    |
|---------|--------------|-------------|-----------|
| 1       | Rahul Sharma | 82998.00    | Completed |
| 2       | Priya Patel  | 1299.00     | Completed |
| 3       | Amit Kumar   | 5598.00     | Completed |
| 4       | Rahul Sharma | 2999.00     | Completed |
| 6       | Vikram Singh | 3398.00     | Completed |

## 5.6 DCL and TCL Operations

-- 1. Remove dependencies and privileges from the previous run

```
DROP OWNED BY admin_user, sales_user, viewer_user, order_manager, payment_clerk;
```

-- 2. Drop roles safely now that dependencies are cleared

```
DROP ROLE IF EXISTS admin_user, sales_user, viewer_user, order_manager, payment_clerk;
```

-- 3. Create roles

```
CREATE ROLE admin_user;
```

```
CREATE ROLE sales_user;
```

```
CREATE ROLE viewer_user;
```

```
CREATE ROLE order_manager;
```

```
CREATE ROLE payment_clerk
```

### Q16–Q17. DCL — GRANT and REVOKE.

```
GRANT SELECT ON Product TO app_user;
```

```
REVOKE SELECT ON Product FROM app_user;
```

### Q18–Q20. TCL — COMMIT, SAVEPOINT and ROLLBACK.

```
START TRANSACTION;
```

```
UPDATE Product SET Stock = 30 WHERE ProductID = 1;
```

```
SAVEPOINT after_stock_update;
```

```
UPDATE Product SET Price = 99999 WHERE ProductID = 1;
```

-- the second update was entered in error

```
ROLLBACK TO SAVEPOINT after_stock_update;
```

```
COMMIT;
```

2 rows affected

## CHAPTER 6: MODULES AND FUNCTIONALITIES

Table 6.1: Modules and Their Functionalities

| S.No | Module Name           | Functionality                                                                                                                                                                   | Tables Involved           |
|------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| 1    | Customer Management   | Handles customer registration, login, profile management (Name, Email, Phone, Address, Password). Allows customers to view and update their personal details.                   | Customer                  |
| 2    | Category Management   | Manages product categories (Electronics, Clothing, Books, Home Appliances, Sports, etc.). Supports adding, viewing, and organizing categories with descriptions.                | Category                  |
| 3    | Product Management    | Manages the product catalog – add/edit/view products with Name, Description, Price, Stock, and Category. Supports stock updates and product search/filter by price or category. | Product, Category         |
| 4    | Order Management      | Allows customers to place orders. Tracks OrderID, CustomerID, OrderDate, Status (Pending, Confirmed, Shipped, Delivered), and TotalAmount. Supports order status updates.       | Order, Customer           |
| 5    | Order Item Management | Manages individual items within an order (Product, Quantity, UnitPrice). Calculates line totals and maintains the relationship between Orders and Products.                     | OrderItem, Order, Product |
| 6    | Payment Management    | Handles payment processing for orders. Records PaymentID, Amount, Method (UPI, Credit Card, Debit Card, Cash on Delivery), PaymentDate, and Status (Pending/Completed).         | Payment, Order            |

The database supports six functional modules. Each module is a group of related SQL operations performed on a defined set of tables.

The OrderItem module is the centre of the design. Because it carries the foreign keys of both Order and Product, it converts the many-to-many association into two one-to-many relationships, and because it stores UnitPrice at the time of purchase it preserves historical pricing even if the catalogue price later changes.

## CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

The principal reports produced by the system are obtained from a single SELECT statement against a view or a join, replacing what was previously a manual compilation from several files.

### Report 1 — Product catalogue with category names (INNER JOIN / view)

```
SELECT * FROM vw_ProductCatalog;
```

| name               | price    | categoryname    |
|--------------------|----------|-----------------|
| Samsung Galaxy S24 | 74999.00 | Electronics     |
| Sony WH-1000XM5    | 29990.00 | Electronics     |
| Men Formal Shirt   | 1299.00  | Clothing        |
| Women Summer Dress | 1899.00  | Clothing        |
| Python Programming | 599.00   | Books           |
| Atomic Habits      | 450.00   | Books           |
| Air Fryer 4L       | 4999.00  | Home Appliances |
| Mixer Grinder      | 3499.00  | Home Appliances |
| Yoga Mat           | 899.00   | Sports          |
| Dumbbell Set 10kg  | 2499.00  | Sports          |
| Wireless Earbuds   | 2999.00  | Electronics     |
| iPhone 15          | 79999.00 | Electronics     |

**Fig. 7.1: Product Catalog with Category (INNER JOIN)**

### Report 2 — Order statistics with aggregate functions

```
SELECT SUM(TotalAmount), MAX(TotalAmount), COUNT(OrderID),  
AVG(TotalAmount), MIN(TotalAmount) FROM "Order";
```

| sum       | max      | count | avg                | min     |
|-----------|----------|-------|--------------------|---------|
| 103190.00 | 82998.00 | 7     | 14741.428571428571 | 1299.00 |

**Fig. 7.2: Order Summary with Aggregate Functions**

## CHAPTER 8: TESTING AND DATA VALIDATION

**Table 8.1: Test Cases and Results**

| TC No. | Test Case Description          | Input / Action                           | Expected Result            | Actual Result              | Status |
|--------|--------------------------------|------------------------------------------|----------------------------|----------------------------|--------|
| TC-01  | Primary key uniqueness         | Insert Customer with existing CustomerID | Duplicate entry error      | Duplicate entry error      | Pass   |
| TC-02  | Unique email                   | Insert Customer with existing email      | Unique violation error     | Unique violation error     | Pass   |
| TC-03  | NOT NULL enforcement           | Insert Customer without Name             | Not-null violation         | Not-null violation         | Pass   |
| TC-04  | CHECK on Price                 | Insert Product with Price = -10          | Check constraint violated  | Check constraint violated  | Pass   |
| TC-05  | CHECK on Quantity              | Insert OrderItem with Quantity = 0       | Check constraint violated  | Check constraint violated  | Pass   |
| TC-06  | Referential integrity (child)  | Insert Product with invalid CategoryID   | Foreign key violation      | Foreign key violation      | Pass   |
| TC-07  | Referential integrity (parent) | Delete Category that has products        | Delete rejected by FK      | Delete rejected by FK      | Pass   |
| TC-08  | DEFAULT value                  | Insert Order without Status              | Status stored as 'Pending' | Status stored as 'Pending' | Pass   |
| TC-09  | Valid INSERT and SELECT        | Insert valid Category and SELECT         | Row inserted and retrieved | Row inserted and retrieved | Pass   |
| TC-10  | INNER JOIN correctness         | Join Product and Category                | Matching rows returned     | Matching rows returned     | Pass   |
| TC-11  | Aggregate accuracy             | SUM/AVG on Order TotalAmount             | Correct aggregate values   | Correct aggregate values   | Pass   |
| TC-12  | View query correctness         | SELECT from vw_ProductCatalog            | Correct joined rows        | Correct joined rows        | Pass   |

## CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

### 9.1 Repository Details

GitHub Repository Link: <https://github.com/Dharanihey/online-shopping-system-dbms->

### 9.2 Team Responsibilities and Contribution

**Table 9.1: Team Responsibilities**

| S.No | Student Name   | Roll Number | Role / Responsibility                                                                            |
|------|----------------|-------------|--------------------------------------------------------------------------------------------------|
| 1    | D. Giri Babu   | 25B11AI242  | Wrote complete DDL scripts (tables + all constraints); Aggregate functions & Views; Prepared PPT |
| 2    | E. Dharani     | 25B11AI291  | Wrote DML scripts (sample data insertion); Joins & CRUD queries; Prepared PPT                    |
| 3    | CH. Sujatha    | 25B11AI211  | Took screenshots of all query outputs; Created PDFs and compiled project documentation           |
| 4    | M. Vivek Surya | 25B11AI773  | Designed ER Diagram & Relational Schema; Prepared documentation                                  |

### 9.3 Review-wise Progress

**Table 9.2: Review-wise Progress Record**

| Review      | Date         | Deliverables Presented                                                       | Remarks of the Guide |
|-------------|--------------|------------------------------------------------------------------------------|----------------------|
| Review – I  | As scheduled | Entities, Attributes, ER Diagram, Relationships, Schema, Keys, Normalization | Completed            |
| Review – II | As scheduled | DDL, DML, CRUD, Joins, Aggregates, Views, Testing, Documentation             | Completed            |

## CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

### 10.1 Conclusion

The Online Shopping System was designed, implemented and tested as a normalised relational database. Six tables were created — Customer, Category, Product, Order, OrderItem and Payment — with five foreign-key relationships connecting them. The design was carried from an ER diagram through a relational schema to the Third Normal Form, removing repeating groups at 1NF, partial dependencies at 2NF and transitive dependencies at 3NF.

All five objectives stated in Section 1.4 were met. The database was implemented in PostgreSQL with primary keys, foreign keys and NOT NULL, UNIQUE, CHECK and DEFAULT constraints; every table was populated with sample data; SQL operations covering retrieval, joins, aggregates with GROUP BY and HAVING, nested subqueries, views, DCL and TCL were executed; and twelve test cases confirmed that the database rejects duplicate keys, out-of-range values and orphan records.

The main outcome is that a question which previously required manual compilation from several spreadsheets — for example the total revenue across all orders, or the products priced above the catalogue average — is now answered by a single SQL statement against data that is stored exactly once.

### 10.2 Limitations of the Present Work

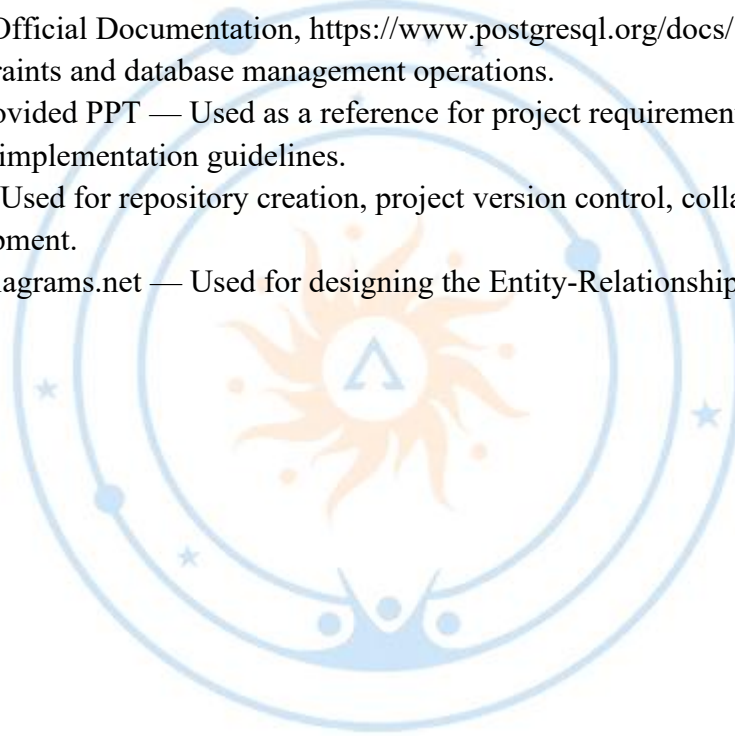
- The system has no front-end application; all interaction is through SQL, so it is usable only by someone who knows the schema.
- Real-time payment gateway integration, user authentication and role-based access control are outside the present scope.
- Product search, filtering UI and recommendation features are not included.
- The sample data set is small; performance behaviour under a realistic load of several thousand products and orders has not been measured.
- Backup and recovery procedures were not configured for the project database.

### 10.3 Future Enhancements

1. Develop a web front-end so that customers and store administrators can use the system without writing SQL.
2. Add user authentication and secure login with role-based access control.
3. Introduce online payment gateway integration.
4. Add product search, filtering and recommendation features.
5. Implement stored procedures for repetitive operations such as bulk stock updates, and triggers to write an audit row whenever an order status is modified.
6. Create indexes on Order(CustomerID), OrderItem(OrderID) and Product(CategoryID) and study the change in query plans as the data volume grows.

## REFERENCES

- ✓ Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
- ✓ Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
- ✓ Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
- ✓ PostgreSQL Official Documentation, <https://www.postgresql.org/docs/> — Used for SQL syntax, constraints and database management operations.
- ✓ 6. Faculty-Provided PPT — Used as a reference for project requirements, database design concepts and implementation guidelines.
- ✓ 7. GitHub — Used for repository creation, project version control, collaboration and team-based development.
- ✓ 8. draw.io / diagrams.net — Used for designing the Entity-Relationship diagram.



## APPENDIX A – COMPLETE SQL SCRIPT

Diagrams/

└─ er-diagram.png.jpeg

screenshots/

└─ Aggregate query outputs.pdf

└─ Crud query outputs.pdf

└─ Joins query outputs.pdf

└─ ddl, dml queries outputs.pdf

└─ views query outputs.pdf

sql/

└─ queries.sql/

| └─ AGGREGATE.sql

| └─ CRUD.sql

| └─ JOINS.sql

| └─ VIEWS.sql

└─ ddl.sql

└─ dml.sql

README.md

---

